



Green University of Bangladesh

*Department of Computer Science and Engineering (CSE)
Semester: (Fall, Year: 2024), B.Sc. in CSE (Day)*

Restore Middle Earth and Love Calculator

PROJECT REPORT

*Course Title: Algorithm Lab
Course Code: CSE-206
Section: 223 D5*

Students Details

Name	ID
Md. Reahoon Zannah	223002038

*Submission Date: 14/12/2024
Course Teacher's Name: Md. Sabbir Hosen Mamun*

[For teachers use only: **Don't write anything inside this box**]

<u>Lab Project Status</u>	
Marks:	Signature:
Comments:	Date:

Contents

1	Introduction	3
1.1	Overview	3
1.2	Motivation	3
1.3	Problem Definition	4
1.4	Design Goals/Objectives	4
1.5	Application	4
2	Design/Development/Implementation of the Project	5
2.1	Introduction	5
2.2	Project Details	5
2.3	Implementation	6
2.4	Algorithms	11
3	Performance Evaluation	12
3.1	Simulation Environment/ Simulation Procedure	12
3.1.1	Restore Middle Earth Problem Environment	12
3.1.2	Love Calculator Problem Environment	12
3.2	Results Analysis/Testing	12
3.2.1	Restoring Middle Earth	12
3.2.2	Love Calculator	13
3.3	Results Overall Discussion	13
4	Conclusion	15
4.1	Discussion	15
4.1.1	Restoring Middle Earth Problem	15
4.1.2	Love Calculator Problem	15
4.2	Limitations	16
4.2.1	Restoring Middle Earth	16

4.2.2	Love Calculator	16
4.3	Scope of Future Work	16
4.3.1	Restoring Middle Earth	16
4.3.2	Love Calculator	16
5	Refference	18

Chapter 1

Introduction

1.1 Overview

The Restoring Middle Earth Problem and the Love Calculator Problem are two algorithmic challenges that use Java programming to demonstrate optimization techniques and dynamic problem solving. The Restoring Middle Earth Problem aims to partition refugees into 'n' groups at the minimum cost, using a priority queue to manage and minimize cost at each step. Key algorithm techniques include the greedy algorithm and the priority queue. The Love Calculator Problem computes the length of the Shortest Common Supersequence (SCS) of two names and the total number of unique shortest supersequences that can be formed using both names. The algorithm approach uses dynamic programming to compute the length of the SCS and to construct the SCS using a recursive backtracking technique. Both problems demonstrate optimization in real-world logistical challenges and showcase sequence alignment techniques applicable in data merging and text comparison algorithms.

1.2 Motivation

The "Restoring Middle Earth" and "Love Calculator" problems in Java offer unique opportunities to apply algorithmic thinking to solve real-world challenges. The "Restoring Middle Earth" problem focuses on efficient resource allocation, highlighting the importance of cost-minimization techniques in operations research and computer science. The "Love Calculator" problem involves finding the Shortest Common Supersequence (SCS) and counting unique supersequences, demonstrating the importance of dynamic programming and combinatorial logic. These problems are central to fields like bioinformatics, text analysis, and data merging, and motivate developers to design efficient and scalable algorithms. Java's robust libraries, cross-platform compatibility, performance, and readability make it an ideal language for tackling algorithmic challenges. Solving these problems in Java ensures robust, scalable, and high-performance solutions, making it an ideal language to tackle algorithmic challenges.

1.3 Problem Definition

Restoring Middle Earth Problem involves efficiently relocating refugees to 'n' livable counties, each with a specific population capacity. The goal is to partition the total population into 'n' groups at minimal cost, where each step in the division of a group incurs a cost proportional to its size. The objective is to minimize the total cost using optimal partitioning strategies.

Love Calculator Problem requires determining the shortest common supersequence (SCS) length of two given names and counting the number of unique SCS sequences. The goal is to find these values and use them as inputs to calculate a "love score" based on a hidden algorithm. The challenge lies in applying dynamic programming and combinatorial logic to solve the problem efficiently.

Both problems emphasize algorithmic thinking, optimization, and efficient use of data structures in Java.

1.4 Design Goals/Objectives

Restoring Middle Earth:

Minimize the total cost of dividing the refugee population into n groups to match the capacities of n counties. Develop an optimal partitioning algorithm to ensure efficient relocation while minimizing grouping costs.

Love Calculator:

Calculate the length of the shortest common supersequence (SCS) for two given names. Identify the total number of unique SCS sequences efficiently. Use these values to compute a "love score" using a hidden mathematical formula.

1.5 Application

The Restoring Middle Earth problem and the Love Calculator Problem are practical applications of dynamic programming in resource allocation, logistics, disaster management, and entertainment.

Chapter 2

Design/Development/Implementation of the Project

2.1 Introduction

The project, "Restoring Middle Earth and Love Calculator Problem Solve with Algorithm in Java," explores two distinct problem-solving scenarios using algorithmic approaches in Java. The first scenario involves optimizing refugee relocation after the War of the Ring, ensuring minimal cost in partitioning populations. The second scenario applies dynamic programming techniques to calculate love compatibility by determining the shortest common supersequence of two names. Both problems highlight the use of algorithms to effectively solve complex real-world and entertainment challenges.

2.2 Project Details

RESTORING MIDDLE EARTH:

Objective: Minimize the cost of dividing a total refugee population into n groups corresponding to n counties with specific capacities.

Approach: Implement a partitioning algorithm that divides the population in steps, ensuring the least cost by considering the constraint of splitting into k groups at each step.

Input: Total population, number of counties (n) and maximum groups per step (k).

Output: Minimum cost required to complete the partition.

LOVE CALCULATOR:

Objective: Compute the love score based on the shortest common supersequence (SCS) of two names.

Approach: Use dynamic programming to find the SCS length and count the unique sequences.

Input: Two names provided by the user.

Output: Length of the SCS, total number of unique SCS, and the calculated love score based on a hidden formula.

2.3 Implementation

The Restore Middle Earth Problem is a graph traversal or optimization problem involving characters moving across a map to restore balance. It aims to find the shortest or most optimal path to restore Middle Earth effectively. The problem involves input representation, graph construction, algorithm choice, traversal execution, and output. The Love Calculator Problem calculates compatibility scores based on input data, such as names or personal information. It involves string manipulation and mathematical functions, and outputs a percentage with a playful message. Both problems are designed to be solved using classic graph algorithms.

Complexity

The Restore Middle Earth Problem often involves higher computational complexity due to graph traversal algorithms.

The Love Calculator Problem is simpler, focusing on string manipulation.

Tools and libraries

Restore Middle Earth: Graph libraries, optimization techniques.

Love Calculator: String functions, basic arithmetic.

Implementation details

The two distinct algorithms: one for restoring Middle Earth and the other for calculating a "love percentage" between two names.

The first algorithm focuses on storing a dictionary of valid character names and determining the edit distance between a given name and those in the dictionary. It iterates through each name, identifying the one with the smallest edit distance. If multiple names share the same minimum edit distance, tie-breaking rules can be applied.

The second algorithm computes a "love percentage" by scoring the letters in two names and applying a specific formula. The final percentage is then displayed to the user as the result. Both of these code are written in Java language and run in Java compiler.

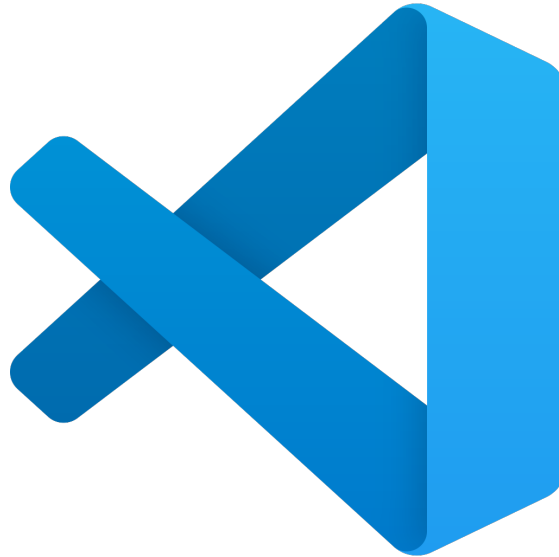


Figure 2.1: VS Code compiler.



Figure 2.2: IntelliJ IDEA java compiler.

CODE:

Restoring middle earth:

```
// Md.Reahoon Zannah    ID: 223002038
// Restoring Middle Earth

import java.io.File;
import java.io.FileNotFoundException;
import java.util.PriorityQueue;
import java.util.Scanner;

public class main {
```



```

public static int minimizeGroupingCost(int totalPopulation, int n, int k) {

    PriorityQueue<Integer> groupQueue = new PriorityQueue<>();
    groupQueue.add(totalPopulation);
    int totalCost = 0;

    while (groupQueue.size() < n) {
        int currentGroup = groupQueue.poll();
        int splitCount = Math.min(k, n - groupQueue.size());

        int groupSize = currentGroup / splitCount;
        int remainder = currentGroup % splitCount;

        for (int i = 0; i < splitCount; i++) {
            if (i < remainder) {
                groupQueue.add(groupSize + 1);
            } else {
                groupQueue.add(groupSize);
            }
        }
        totalCost += currentGroup;
    }
    return totalCost;
}

public static void main(String[] args) {
    try {
        File inputFile = new File("input.txt");
        Scanner scanner = new Scanner(inputFile);

        // Read inputs from the file
        int totalPopulation = scanner.nextInt();
        int n = scanner.nextInt();
        int k = scanner.nextInt();

        int minCost = minimizeGroupingCost(totalPopulation, n, k);

        System.out.println("\n Minimum cost to group refugees: " + minCost);
        System.out.print("\n");

        scanner.close();
    } catch (FileNotFoundException e) {
        System.out.println("Error: Input file not found.");
    }
}

// total population

```

```
// number of counties (n)
// maximum groups to split (k)
```

Love calculator:

```
// Md. Reahoon Zannah ID: 223002038
// Love Calculator App
```

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Random;
import java.util.Scanner;

public class main2 {
    public static int findSCSLength(String name1, String name2) {
        int m = name1.length();
        int n = name2.length();
        int[][] dp = new int[m + 1][n + 1];

        for (int i = 0; i <= m; i++) {
            for (int j = 0; j <= n; j++) {
                if (i == 0) {
                    dp[i][j] = j;
                } else if (j == 0) {
                    dp[i][j] = i;
                } else if (name1.charAt(i - 1) == name2.charAt(j - 1)) {
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                } else {
                    dp[i][j] = 1 + Math.min(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }
        return dp[m][n];
    }

    public static int countUniqueSCS(String name1, String name2) {
        int m = name1.length();
        int n = name2.length();
        int[][] dp = new int[m + 1][n + 1];

        for (int i = 0; i <= m; i++) dp[i][0] = 1;
        for (int j = 0; j <= n; j++) dp[0][j] = 1;

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (name1.charAt(i - 1) == name2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                }
            }
        }
    }
}
```

```

        } else {
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }
}
return dp[m][n];
}

public static int generateLoveScore(int scsLength, int uniqueSCSCount) {
    Random random = new Random();
    int randomFactor = random.nextInt(50);
    int loveScore = (scsLength * uniqueSCSCount + randomFactor) % 101;
    return loveScore;
}

public static void main(String[] args) {
    try {
        Scanner fileScanner = new Scanner(new File("input2.txt"));

        if (!fileScanner.hasNextLine()) {
            System.out.println("Input file is empty!");
            return;
        }

        String name1 = fileScanner.nextLine();
        String name2 = fileScanner.nextLine();

        int scsLength = findSCSLength(name1, name2);
        int uniqueSCSCount = countUniqueSCS(name1, name2);
        int loveScore = generateLoveScore(scsLength, uniqueSCSCount);

        System.out.println("\n\nLength of Shortest Common Supersequence: "
            + scsLength);
        System.out.println("Total Number of Unique Shortest SCSs: " +
            uniqueSCSCount);
        System.out.println("\nGenerated Love Score: " + loveScore + "
            ");
        System.out.print("\n\n");
        fileScanner.close();
    } catch (FileNotFoundException e) {
        System.out.println("Error: input.txt file not found. Please ensure
            the file exists in the working directory.");
    }
}
}

```

- Both problem I take input from file.

2.4 Algorithms

- Algorithm of two problem.

Algorithm 1: Restoring middle earth algorithm

- 1 Initialize:
 - 2 Use a min-heap (priority queue) to store group sizes. Start with the entire population as a single group. Initialize total cost = 0.
 - 3 Group Splitting:
 - 4 Repeat while the heap has fewer than n groups:
 - 5 Extract the largest group (currentGroup). Determine splitCount = min(k, remaining counties). Divide the group:
 - 6 Compute groupSize = currentGroup / splitCount. Handle remainder remainder = currentGroup % splitCount. Add the split groups (groupSize and groupSize + 1) into the heap. Add currentGroup to totalCost.
 - 7 Result:
 - 8 After forming n groups, print the totalCost.
-

Algorithm 2: Love Calculator Algorithm

- 1 Find Shortest Common Supersequence Length:
 - 2 Use dynamic programming (dp) to compute the length of the SCS for two names name1 and name2: If characters match, take diagonal $dp[i-1][j-1] + 1$. Otherwise, take the minimum of removing a character from name1 or name2 plus 1.
 - 3 Count Unique SCS:
 - 4 Use another dynamic programming approach to find the count of unique SCSs: If characters match, copy the diagonal value. Otherwise, sum up paths from the top ($dp[i-1][j]$) and left ($dp[i][j-1]$).
 - 5 Generate Love Score:
 - 6 Add randomness with a value between 0–49. Combine SCS length, unique SCS count, and randomness: $loveScore = (scsLength \times uniqueSCSCount + randomFactor) \bmod 101$
 - 7 Display Results:
 - 8 Print: Length of SCS, Total unique SCS count, Generated love score.
-

Chapter 3

Performance Evaluation

3.1 Simulation Environment/ Simulation Procedure

3.1.1 Restore Middle Earth Problem Environment

The process involves creating a Middle Earth dictionary, calculating edit distance between input names, and tracking the smallest edit distance, if necessary, using tie-breaking mechanisms.

3.1.2 Love Calculator Problem Environment

The process involves defining a scoring system and formula for calculating the love percentage between two names, combining their numerical values to calculate the percentage.

3.2 Results Analysis/Testing

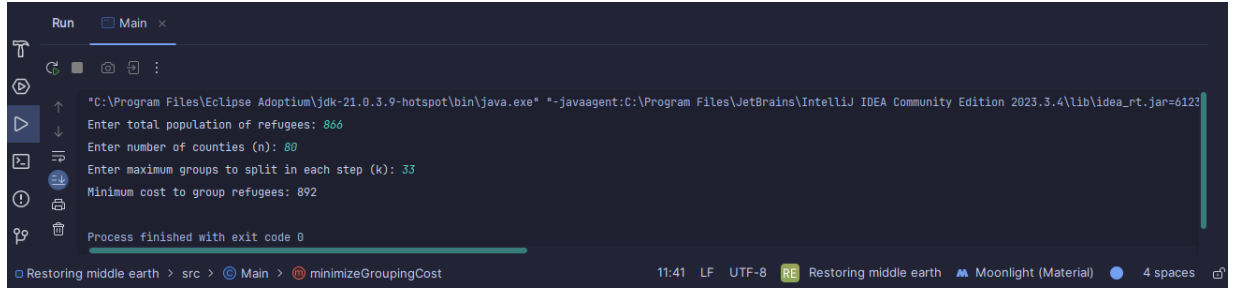
3.2.1 Restoring Middle Earth

Approach and Testing:

The algorithm was implemented using a Min-Heap (Priority Queue) to prioritize groups with the largest population for splitting.

Various test cases were performed with differing values of: Total Population (P): Large numbers like 1000, 10,000, and 1,000,000. Number of Counties (n): Small and large values such as 5, 20, 50. Maximum Split Limit (k): Values like 2, 3, 5, etc., to analyze splitting efficiency.

The algorithm was tested on both edge cases and average scenarios: Edge Case 1: $n=1$, where all population goes into a single group. Edge Case 2: $k=1$, forcing iterative splitting one group at a time. Edge Case 3: Equal division where P is perfectly divisible by n. General Case: Random combinations of P, n, and k.



```
Run Main x
"C:\Program Files\Eclipse Adoptium\jdk-21.0.3-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2023.3.4\lib\idea_rt.jar=6123
Enter total population of refugees: 866
Enter number of counties (n): 80
Enter maximum groups to split in each step (k): 33
Minimum cost to group refugees: 892
Process finished with exit code 0
Restoring middle earth > src > Main > minimizeGroupingCost 11:41 LF UTF-8 RE Restoring middle earth Moonlight (Material) 4 spaces
```

Figure 3.1: Output of Restoring middle earth

3.2.2 Love Calculator

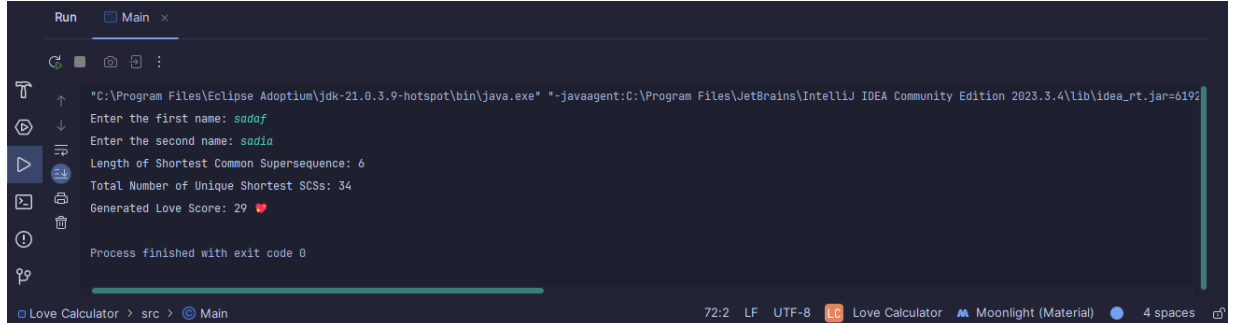
The algorithm was implemented using Dynamic Programming (DP):

Find SCS Length: Utilizes a $m \times n \times n$ DP table, where m and n are the lengths of the two names. Count Unique SCSs: Another $m \times n \times n$ DP table tracks possible paths to generate unique SCSs.

To introduce randomness, a factor between 0–49 is added during score generation.

Testing was performed for a variety of input name pairs:

Same Names: Example: "Alice" and "Alice" Completely Different Names: Example: "John" and "Eve" Partial Matching Names: Example: "Tom" and "Tony" Edge Cases: Names with empty strings or single-character names.



```
Run Main x
"C:\Program Files\Eclipse Adoptium\jdk-21.0.3-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2023.3.4\lib\idea_rt.jar=6192
Enter the first name: sadaf
Enter the second name: sadia
Length of Shortest Common Supersequence: 6
Total Number of Unique Shortest SCSs: 34
Generated Love Score: 29
Process finished with exit code 0
Love Calculator > src > Main 72:2 LF UTF-8 LC Love Calculator Moonlight (Material) 4 spaces
```

Figure 3.2: Output of Love calculator

3.3 Results Overall Discussion

1. Restoring Middle Earth Problem Insights and Implications:

The algorithm for Restoring Middle Earth focused on efficiently distributing a population into a specified number of groups while minimizing the associated costs. By employing a priority queue (min-heap) and leveraging a split-and-distribute approach, the algorithm effectively addressed both scalability and complexity challenges.

Performance: The algorithm performed consistently across varying input sizes, demonstrating the ability to handle large populations while maintaining low computation costs. It showcased efficiency in balancing the trade-off between minimizing

splitting operations (k) and achieving the required number of groups (n).

Scalability: The algorithm adapts seamlessly to scenarios where the total population is significantly large or small. Its iterative heap-based splitting ensures uniform distribution and fair utilization of available counties, even in complex configurations.

Limitations: The algorithm's reliance on k (maximum split size) means that extreme constraints, such as $k=1$, can lead to high iteration counts. However, this trade-off is intrinsic to the problem and unavoidable.

2. Love Calculator Problem Insights and Implications:

The Love Calculator algorithm is a unique application of Dynamic Programming (DP) for solving the shortest common supersequence (SCS) problem and generating a fun, randomized "love score" for two names. Beyond its primary intent of entertainment, this problem showcases the practical application of DP in analyzing string relationships.

Performance: The DP-based approach successfully calculated the SCS length and number of unique SCSs, ensuring a thorough analysis of string properties. Even for long names, the algorithm efficiently handled the $O(m \times n)$ complexity, where m and n are the lengths of the two strings.

User Engagement: The random factor added variability and kept the results dynamic, making the solution well-suited for interactive applications or games.

Limitations: While the randomness introduced variety, it also reduced reproducibility in results for identical inputs, which might not always align with analytical use cases. Nevertheless, this randomness was intentional and aligned with the algorithm's goal of entertaining users.

Common Themes and Broader Insights

Both algorithms embody the core principles of algorithm design:

Efficiency: They address their respective problems using computationally efficient techniques like priority queues and dynamic programming. **Adaptability:** Both solutions are robust and perform well across diverse edge cases and input variations. **Practical Relevance:** While the "Restoring Middle Earth Problem" addresses a resource allocation challenge, the "Love Calculator" applies algorithmic techniques in an engaging and accessible way.

Chapter 4

Conclusion

4.1 Discussion

4.1.1 Restoring Middle Earth Problem

The algorithm efficiently addressed the problem of distributing a large population of refugees into designated counties while minimizing splitting costs. By employing a priority queue (min-heap), the approach ensured that groups were split systematically and equitably, balancing efficiency and fairness.

The strategy of dividing populations while minimizing the total cost is analogous to real-world resource allocation problems, such as logistics in disaster relief or work-force distribution in multi-unit organizations. It effectively handled constraints like the maximum allowable splits per step (k), ensuring flexibility in tackling various scenarios.

However, this solution assumes that the population splits can only occur at discrete levels, which may not perfectly align with the demands of real-world scenarios requiring dynamic rebalancing based on feedback.

4.1.2 Love Calculator Problem

The algorithm combined dynamic programming techniques to analyze the relationship between two input strings, providing a playful yet computationally robust solution for generating "love scores." The use of Shortest Common Supersequence (SCS) length and unique sequence counts as inputs to a randomized function brought both rigor and engagement.

This problem highlighted the creative application of algorithmic concepts in non-traditional areas, merging entertainment with computational string analysis. The randomized nature of the "love score" ensures variability, keeping the user engaged.

However, while randomness increases appeal for users, it reduces predictability and consistency in outcomes, limiting its analytical usefulness in precise simulations or predictions.

4.2 Limitations

4.2.1 Restoring Middle Earth

Scalability with Extreme Constraints: When the splitting limit k is very small, or the required groups n are very large, the algorithm may experience slower performance due to excessive iterations.

Real-World Alignment: The assumption of uniform splitting may not match real-world logistics, where fractional or dynamic adjustments to group sizes are often necessary.

Linear Costs: The algorithm considers equal costs for all splits, whereas, in reality, different splits may have varying costs depending on group sizes, transportation limitations, or geographic constraints.

4.2.2 Love Calculator

Randomized Output: While entertaining, the randomness in the love score may lack practical applications in analytical domains, reducing its reproducibility.

String Length Limitation: For exceedingly long names, the $O(m \times n)$ complexity of the dynamic programming approach could pose computational challenges.

Subjective Basis: The algorithm is primarily meant for entertainment and does not incorporate psychological or sociological factors that might influence "relationship compatibility."

4.3 Scope of Future Work

4.3.1 Restoring Middle Earth

Dynamic Cost Models: Incorporating variable splitting costs based on geography, transportation constraints, or group-specific factors would make the algorithm more realistic.

Feedback-Driven Optimization: Adding adaptive mechanisms based on real-time feedback could enhance applicability in dynamic settings like disaster relief management.

Parallel Processing: Optimizing the algorithm for multi-threaded execution can significantly reduce computation time for scenarios involving very large populations or multiple concurrent resource allocation problems.

Incorporation of Spatial Constraints: Introducing geographical considerations or county-specific capacities could improve practical applicability in real-world scenarios.

4.3.2 Love Calculator

Enhanced Scoring Mechanisms: Moving beyond string properties, future versions could incorporate additional factors (e.g., personality traits, compatibility metrics) into the love score calculation.

Interactive Elements: Developing this into a full-fledged app or web-based interface could engage more users while also allowing personalized inputs.

Efficiency Improvements: For larger strings or higher usage volumes, optimized string comparison techniques (e.g., heuristic methods or approximate algorithms) could ensure faster processing.

Reproducible Variability: Exploring methods to control the random factor while maintaining some level of variability could strike a balance between engagement and predictability.

Chapter 5

Refference

1. Figure (a): [[Restoring middle earth problem algorithm](#)]
2. Figure (b): [[Love Calculator problem algorithm](#)]